Exercise 1: Save work with commits

In this exercise, you will step through several scenarios that involve the use of Visual Studio Code to stage and commit changes.

When you make changes to your files, Git will record the changes in the local repository. You can select the changes that you want to commit by staging them. Commits are always made against your local Git repository, so you don't have to worry about the commit being perfect or ready to share with others. You can make more commits as you continue to work and push the changes to others when they are ready to be shared.

Git commits consists of the following:

- The file(s) changed in the commit. Git keeps the contents of all file changes in your repo in the commits. This keeps it fast and allows intelligent merging.
- A reference to the parent commit(s). Git manages your code history using these references.
- A message describing a commit. You give this message to Git when you create the commit. It's a good idea to keep this message descriptive, but to the point.

Task 1: Commit changes

In this task, you will use Visual Studio Code to commit changes.

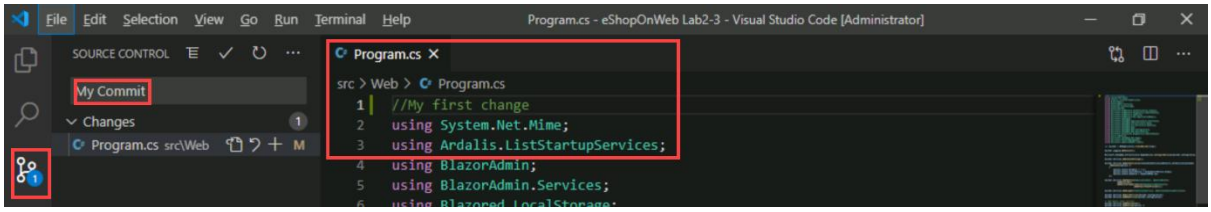
1. In the Visual Studio Code window, at the top of the vertical toolbar, select the **EXPLORER** tab, navigate to the **/eShopOnWeb/src/Web/Program.cs** file and select it. This will automatically display its content in the details pane.
2. On the first line add the following comment:

c#

```
// My first change
```

Note: It doesn't really matter what the comment is since the goal is just to make a change.

3. Press **Ctrl+S** to save the change.
4. In the Visual Studio Code window, select the **SOURCE CONTROL** tab to verify that Git recognized the latest change to the file residing in the local clone of the Git repository.
5. With the **SOURCE CONTROL** tab selected, at the top of the pane, in the textbox, type **My commit** as the commit message and press **Ctrl+Enter** to commit it locally.



6. If prompted whether you would like to automatically stage your changes and commit them directly, click **Always**.

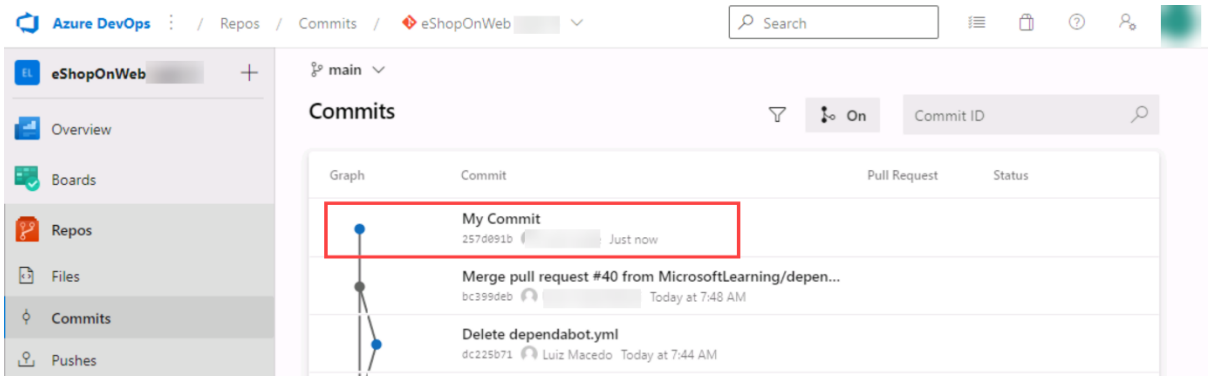
Note: We will discuss **staging** later in the lab.

7. In the lower left corner of the Visual Studio Code window, to the right of the **main** label, note the **Synchronize Changes** icon of a circle with two vertical arrows pointing in the opposite directions and the number **1** next to the arrow pointing up. Click the icon and, if prompted, whether to proceed, click **OK** to push and pull commits to and from **origin/main**.

Exercise 2: Review commits

In this task, you will use the Azure DevOps portal to review commits.

1. Switch to the web browser window displaying the Azure DevOps interface.
2. In the vertical navigational pane of the Azure DevOps portal, in the **Repos** section, select **Commits**.
3. Verify that your commit appears at the top of list.



Task 3: Stage changes

In this task, you will explore the use of staging changes by using Visual Studio Code. Staging changes allows you to selectively add certain files to a commit while passing over the changes made in other files.

1. Switch back to the **Visual Studio Code** window.
2. Update the open **Program.cs** class by changing the first comment with the following, and saving the file.

c#

//My second change

3. In the Visual Studio Code window, switch back the **EXPLORER** tab, navigate to the **/eShopOnWeb/src/Web/Constants.cs** file and select it. This will automatically display its content in the details pane.

4. Add to the **Constants.cs** file a comment on the first line and save the file.

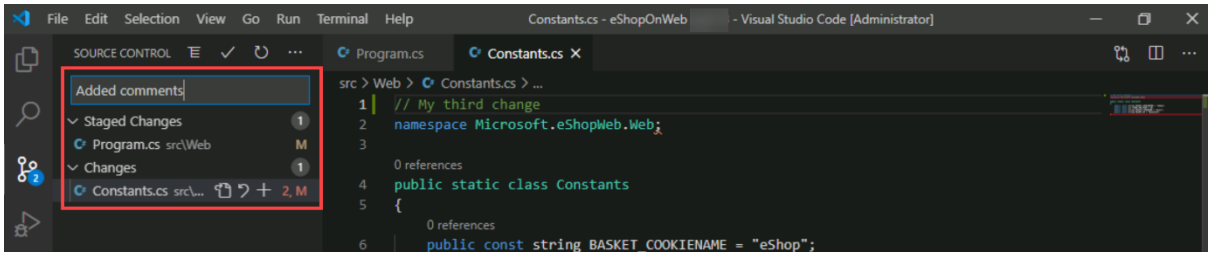
c#

// My third change

5. In the Visual Studio Code window, switch to the **SOURCE CONTROL** tab, hover the mouse pointer over the **Program.cs** entry, and click the plus sign on the right side of that entry.

Note: This stages the change to the **Program.cs** file only, preparing it for commit without **Constants.cs**.

6. With the **SOURCE CONTROL** tab selected, at the top of the pane, in the textbox, type **Added comments** as the commit message.



7. At the top of the **SOURCE CONTROL** tab, click the ellipsis symbol, in the drop-down menu, select **Commit** and, in the cascading menu, select **Commit Staged**.

8. In the lower left corner of the Visual Studio Code window, click the **Synchronize Changes** button to synchronize the committed changes with the server and, if prompted, whether to proceed, click **OK** to push and pull commits to and from **origin/main**.

Note: Note that since only the staged change was committed, the other change is still pending to be synchronized.

Exercise 3: Review history

In this exercise, you will use the Azure DevOps portal to review history of commits.

Git uses the parent reference information stored in each commit to manage a full history of your development. You can easily review this commit history to find out when file changes were made and determine differences between versions of your code using the terminal or

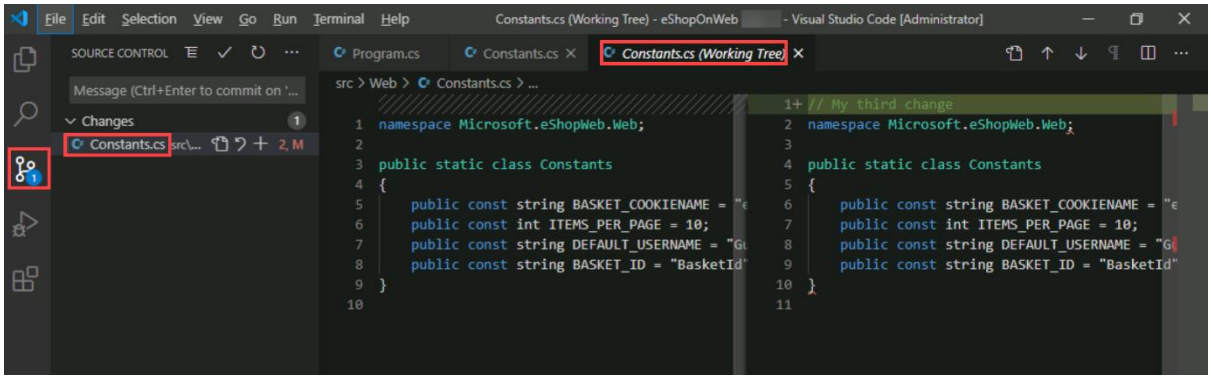
from one of the many available Visual Studio Code extensions. You can also review changes by using the Azure DevOps portal.

Git's use of the **Branches and Merges** feature works through pull requests, so the commit history of your development doesn't necessarily form a straight, chronological line. When you use history to compare versions, think in terms of file changes between two commits instead of file changes between two points in time. A recent change to a file in the main branch may have come from a commit created two weeks ago in a feature branch that was merged yesterday.

Task 1: Compare files

In this task, you will step through commit history by using the Azure DevOps portal.

1. With the **SOURCE CONTROL** tab of the Visual Studio Code window open, select **Constants.cs** representing the non-staged version of the file.

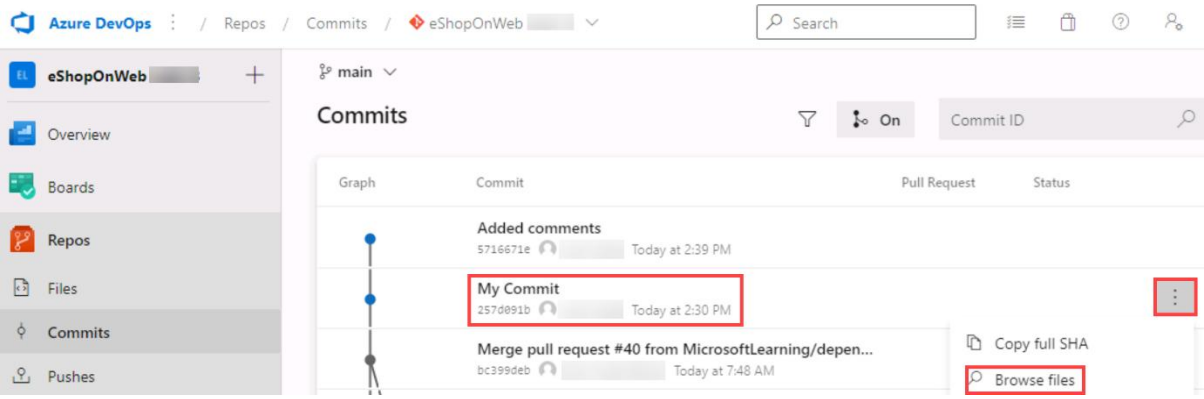


Note: A comparison view is opened to enable you to easily locate the changes you've made. In this case, it's just the one comment.

2. Switch to the web browser window displaying the **Commits** pane of the **Azure DevOps** portal to review the source branches and merges. These provide a convenient way to visualize when and how changes were made to the source.

3. Scroll down to the **My commit** entry (pushed before) and hover the mouse pointer over it to reveal the ellipsis symbol on the right side.

4. Click the ellipsis, in the dropdown menu, select **Browse Files**, and review the results.



Note: This view represents the state of the source corresponding to the commit, allowing you to review and download each of source files.